



Dokumentation

ENTWICKLUNG MOBILER ANWENDUNGEN

HOCHSCHULE WORMS,
SOMMERSEMESTER 2026

VORGELEGT VON

- LUCAS ISSELHARD (MATR.: 677388)
- JONAS HORWEDEL (MATR.: 677359)

ABGABE DER ARBEIT: 24.07.2026

VORGELEGT BEI: LAURA MOST

1. Einleitung.....	2
1.1 Projektbeschreibung.....	2
1.2 Ziel des Projekts	2
1.3 Installationsanweisung	2
1.3.1 Voraussetzungen	3
1.3.2 Repository klonen	3
1.3.3 In Projektverzeichnis wechseln	3
1.3.4 Abhängigkeiten installieren	3
1.3.5 App ausführen	3
1.4 Test-Account.....	3
1.4.1 App.....	3
1.4.2 Supabase.....	3
1.5 User Stories	3
Master-Passwort anlegen	4
App entsperren	4
Vaults verwalten	5
Einträge verwalten	5
Verschlüsseltes Cloud-Backup.....	6
Design Umschaltung.....	7
Bilder auf Geräten speichern	7
Vaults exportieren/teilen (geplant)	8
1.6 Wireframes	8
1.6.1 Setup.....	9
1.6.2 Vault-List	10
1.6.3 Settings.....	10
1.6.4 Cloud Log-In.....	11
1.6.5 Vault-Details	11
1.6.6 Export	11
2. Tools	12
2.1 Programmiersprache und verwendete Version	13
2.2 Entwicklungsumgebungen	13
2.3 APIs.....	13

2.4 Supabase.....	13
2.4.1 Datenbank	14
2.4.2 Policies.....	14
2.4.3 Storage	15
2.5 Wichtige Dependencies	15
Architektur & State Management.....	16
Sicherheit & Verschlüsselung	16
Authentifizierung.....	16
Lokale Datenhaltung.....	16
Sonstige Bibliotheken.....	16
Testing	16
Geplante Features	16
2.6 Quellen und Lizenzen	16
Grafische Assets.....	17
Icons.....	17
Open-Source Bibliotheken.....	17
3. Projektarchitektur	17
3.1 Beschreibung der gesamten Architektur	18
3.2 Use-Case Diagramm.....	18
3.3 Klassendiagramm.....	19
4. Testen.....	20
4.1 Unit Tests.....	21
VaultCubitTest (5 Testfälle).....	21
EncryptionServiceTest (3 Testfälle).....	21
4.2 Manuelle Testfälle	21
4.3 Häufige Probleme und Lösungen.....	22
5. Aktueller Stand	23
6. Ausblick.....	24
6.1 Weiterentwicklung.....	25
7. Fazit	25
8. Quellen.....	26

1. Einleitung

1.1 Projektbeschreibung

Die App “MinVault” ist eine mobile Sicherheits-App zur Verwaltung und geschützten Speicherung von sensiblen Daten. Sie erlaubt es Nutzern sogenannte Vaults (deutsch *Tresore*) zu erstellen und darin Bilder, Notizen, Passwörter und andere Dokumente verschlüsselt abzuspeichern.

1.2 Ziel des Projekts

Ziel des Projekts ist die Entwicklung einer mobilen Anwendung, mit der Nutzer sensible Daten wie Passwörter, private Notizen, Fotos und Dokumente sicher auf ihrem eigenen Gerät speichern können. Dabei sollen sie nicht auf den Schutz gewöhnlicher Notiz- oder Foto-Apps angewiesen sein.

MinVault setzt dabei auf ein konsequentes Zero-Knowledge-Prinzip: Alle Inhalte werden ausschließlich auf dem Gerät ver- und entschlüsselt.

Die dafür benötigten Schlüssel verlassen das Gerät zu keinem Zeitpunkt.

Die App ist als Offline-First-Anwendung aufgebaut und funktioniert vollständig ohne Internetverbindung. Zusätzlich kann eine optionale Cloud-Anbindung über Supabase genutzt werden, um verschlüsselte Backups zu erstellen.

Dabei erhält auch der Anbieter keinen Zugriff auf die unverschlüsselten Daten.

1.3 Installationsanweisung

1.3.1 Voraussetzungen

Flutter SDK: $\geq 3.44.0$ (stable)

Dart SDK: $\geq 3.12.0$ (in der Flutter-SDK-Version enthalten)

Git

1.3.2 Repository klonen

```
git clone https://github.com/xAfterLife/502-EmA
```

1.3.3 In Projektverzeichnis wechseln

```
cd min_vault
```

1.3.4 Abhängigkeiten installieren

```
flutter pub get
```

1.3.5 App ausführen

```
flutter run
```

1.4 Test-Account

1.4.1 App

Für die App selbst ist kein Test-Account erforderlich, da das Setup Off-Line First ist und ein Master-Password bei erstem Starten der Anwendung des Anwenders gesetzt wird.

1.4.2 Supabase

Für die Cloud Funktionalität kann folgender Test-Account verwendet werden:

E-Mail: test@mail.de

Passwort: EmA502 !

1.5 User Stories

Master-Passwort anlegen	
ID	US01
Priorität	Must-have
Beschreibung	Als neuer Nutzer möchte ich beim erstmaligen Starten ein Master-Passwort einrichten, um unbefugten Zugriff auf gespeicherte Daten zu verhindern.
Ergänzungen	Lokale/Offline-Authentifizierung
Zulieferung	/
Abhängigkeiten	/
Definition of Done / Akzeptanzkriterium	Eingabefeld für Passwort und Passwort-Wiederholung, Hinweis auf Eingabe eines ungültigen Passworts, nach Eingabe eines gültigen Passworts Weiterleitung zu Inhalten
Entwickleraufgaben	Eingabeformular erstellen, Lokale Authentifizierung, Weiterleitung zu Inhalten
Offene Fragen	/
Wireframes	Setup-Screen mit Eingabefelder für Passwort und Passwort-Wiederholung, Bestätigungs-Button

App entsperren	
ID	US02
Priorität	Must-have
Beschreibung	Als Nutzer möchte ich durch Eingabe meines Master-Passworts oder alternativ durch Fingerabdruck die App entsperren, um auf die in der App gespeicherten Inhalte zugreifen zu können.
Ergänzungen	Lokale/Offline-Authentifizierung mit Passwort
Zulieferung	/
Abhängigkeiten	US01 - Nutzer muss Master-Passwort eingerichtet haben
Definition of Done / Akzeptanzkriterium	Eingabefeld für Passwort, Passworteingabe unkenntlich machen, Hinweis auf Eingabe eines falschen Passworts,

	Weiterleitung zu Inhalten nach Eingabe des richtigen Passworts
Entwicklertaufgaben	Eingabeformular erstellen, Lokale Authentifizierung, Weiterleitung zu Inhalten
Offene Fragen	Wann soll die App wieder gesperrt werden?
Wireframes	Unlock-Screen mit Eingabefeld für Passwort, Unlock-Button

Vaults verwalten	
ID	US03
Priorität	Must-have
Beschreibung	Als Nutzer möchte ich mehrere Vaults erstellen und löschen können, um meine Inhalte zu organisieren.
Ergänzungen	Vault-Verzeichnisnamen werden im lokalen Speicher unkenntlich gemacht
Zulieferung	/
Abhängigkeiten	US02 – Nutzer muss App entsperrt haben US01 – Nutzer muss Master-Passwort angelegt haben
Definition of Done / Akzeptanzkriterium	Vault-Liste zeigt alle erstellten Vaults an, Vault Informationen angezeigt: Name des Vaults und Anzahl enthaltener Elemente, jeweils Button zum Erstellen/Löschen von Vault vorhanden, verschlüsselte Verzeichnisse auf lokalem Speicher werden angelegt
Entwicklertaufgaben	Vault Liste zeigt Vaults und Vault-Informationen an, Vaults erstellen/löschen, Vault-Verzeichnisnamen verschlüsselt anlegen
Offene Fragen	/
Wireframes	Vault-List-Screen mit Create-Button am unteren Bildschirmrand, Liste zeigt vorhandene Vaults übereinander an, Button für Einstellungen in obere rechte Ecke

Einträge verwalten	
ID	US04

Priorität	Must-have
Beschreibung	Als Nutzer möchte ich Passwörter, Notizen, Bilder und Dateien einem Vault hinzufügen, aus einem Vault löschen und ansehen können, um meine Inhalte sicher aufzubewahren und zu verwalten.
Ergänzungen	/
Zulieferung	/
Abhängigkeiten	US01, US02, US03 – Nutzer muss Vault erstellt haben
Definition of Done / Akzeptanzkriterium	Jeder Vault-Eintrag wird verschlüsselt gespeichert und kann wieder gelöscht werden, Bilder werden zusätzlich mit einem verschlüsselten Thumbnail versehen, Passwörter und Notizen können kopiert werden
Entwickleraufgaben	Daten-Verschlüsselung, Thumbnails erstellen
Offene Fragen	/
Wireframes	Vault-Detail-Screen zeigt Einträge in Liste übereinander an, Button zum Löschen jeden Eintrags, Button am unteren Bildschirmrand zum Hinzufügen von Einträgen

Verschlüsseltes Cloud-Backup	
ID	US05
Priorität	Should-have
Beschreibung	Als Nutzer möchte ich mich optional mit einem Cloud-Konto verbinden und meine Tresore verschlüsselt sichern können, um Datenverlust zu vermeiden.
Ergänzungen	Zero-Knowledge-Prinzip: Es werden nur bereits lokal verschlüsselte Daten in der Cloud gespeichert.
Zulieferung	Supabase Authentifizierung und Storage
Abhängigkeiten	US01, US02, US03
Definition of Done / Akzeptanzkriterium	An- / Abmeldung über Supabase, Vault wird mit bereits verschlüsselten Dateien hoch- bzw. heruntergeladen, Cloud-Synchronisierungsbutton, Cloud Einstellungen
Entwickleraufgaben	Cloud-Backup Button, Supabase Authentication und Storage, Datensynchronisierung, Vault als .zip verpacken

Offene Fragen	/
Wireframes	Toggle-Switch für Cloud-Account in Settings-Screen, Cloud-Backup Button für jeden Vault in Vault-Liste

Design Umschaltung	
ID	US06
Priorität	Nice-To-Have
Beschreibung	Als Nutzer möchte ich einen Darkmode ein- und ausschalten können, um das Aussehen der App an meine Präferenzen anzupassen.
Ergänzungen	/
Zulieferung	/
Abhängigkeiten	US01, US02
Definition of Done / Akzeptanzkriterium	Wechsel zwischen Hell- und Dunkel-Design, die gewählte Einstellung des Darkmodes bleibt nach Neustart bestehen
Entwickleraufgaben	Toggle-Switch in den Einstellungen, Anwendung des gewählten Designs auf gesamte App
Offene Fragen	/
Wireframes	Toggle-Switch im Settings-Screen

Bilder auf Geräten speichern	
ID	US07
Priorität	Should-Have
Beschreibung	Als Nutzer möchte ich Bilder, die sich bereits im Vault befinden, wieder auf dem Gerät speichern, damit diese Bilder wieder frei genutzt werden können.
Ergänzungen	/
Zulieferung	/
Abhängigkeiten	US01, US02, US03, US04 – Bild muss im Vault sein
Definition of Done / Akzeptanzkriterium	Bilder werden im Gerätespeicher unverschlüsselt gespeichert

Entwicklertaufgaben	Bilder entschlüsseln, Bilder unverschlüsselt im Gerätespeicher ablegen
Offene Fragen	/
Wireframes	Save-Button für Bilder hinterlegt

Vaults exportieren/teilen (geplant)	
ID	US08
Priorität	Nice-To-Have
Beschreibung	Als Nutzer möchte ich einen einzelnen Vault unabhängig von der Cloud-Anbindung als Datei exportieren oder teilen können, um ihn manuell zu sichern oder zu übertragen.
Ergänzungen	/
Zulieferung	/
Abhängigkeiten	US01, US02, US03, US04
Definition of Done / Akzeptanzkriterium	Export/Share-Button vorhanden, Vault kann auf Gerät gespeichert oder über andere Apps geteilt werden
Entwicklertaufgaben	Vault wird in Export/Share-Datei zusammengefasst, Export/Share-Datei auf Gerät speichern oder teilen
Offene Fragen	/
Wireframes	Teilen/Export-Button in Vault-Liste

1.6 Wireframes

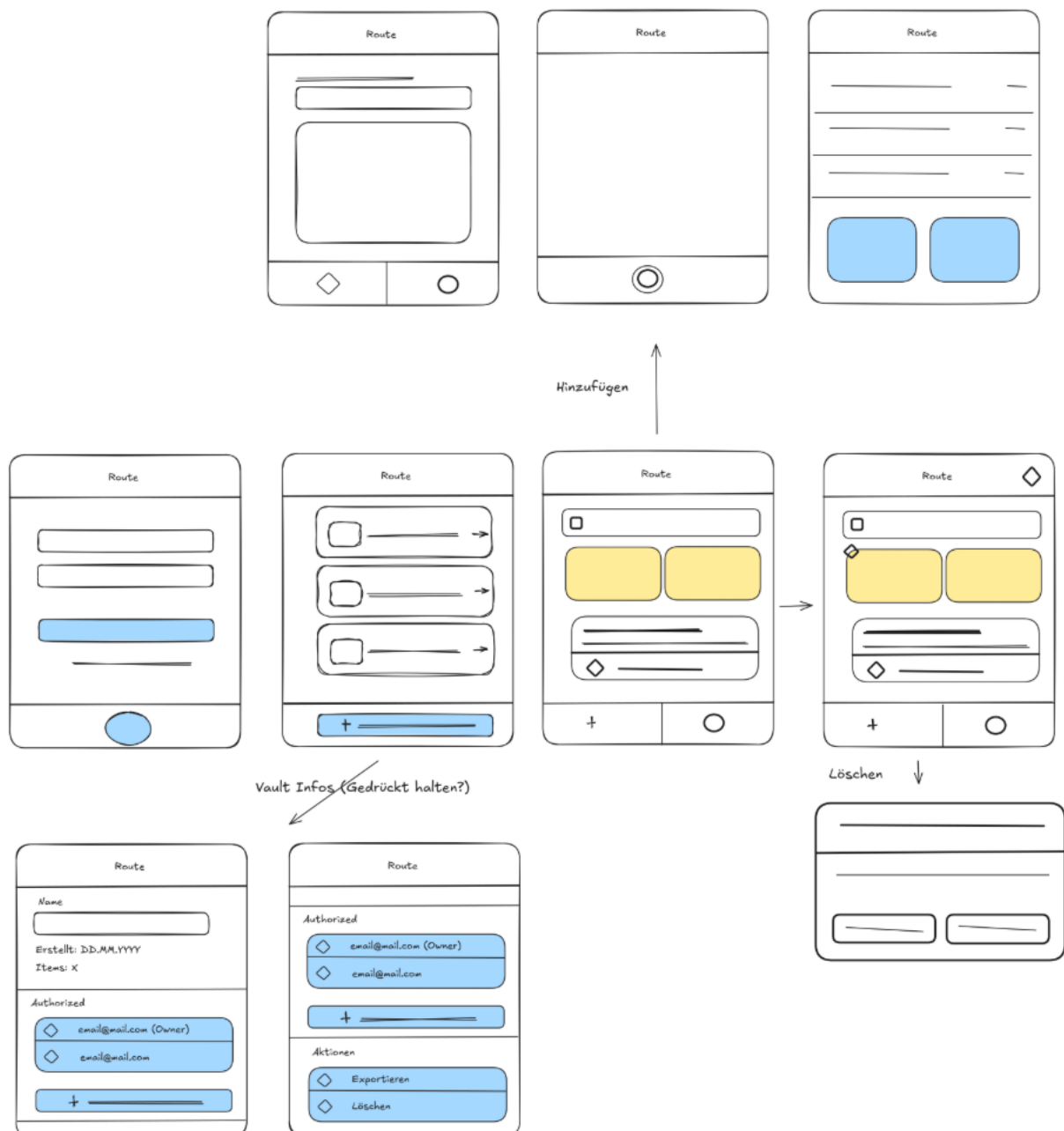
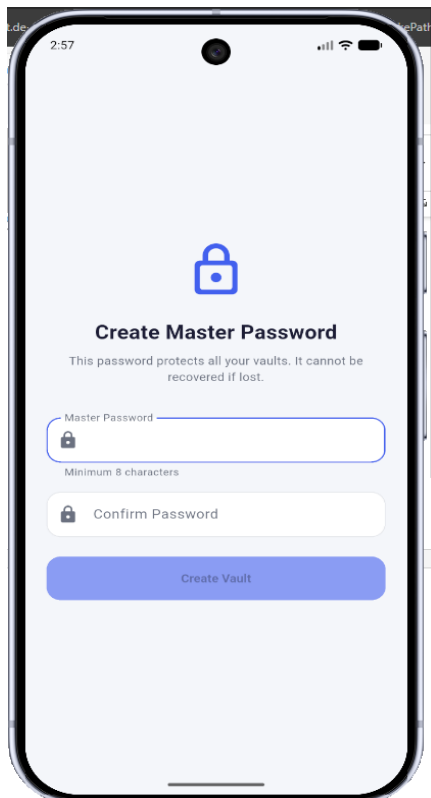


ABBILDUNG 1: INITIALES WIREFRAME

1.6.1 Setup



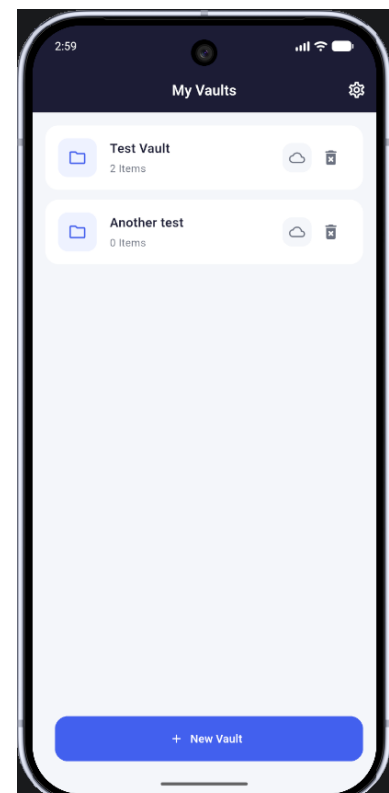
Der Einrichtungsbildschirm ermöglicht die erstmalige Erstellung des Master-Passworts, das den Zugriff auf die Anwendung schützt. Ein kurzer Hinweis informiert darüber, dass dieses Passwort nicht wiederhergestellt werden kann.

Das Passwort wird über zwei Eingabefelder eingegeben und bestätigt. Nach erfolgreicher Prüfung kann die Einrichtung über den Button „Create Vault“ abgeschlossen werden.

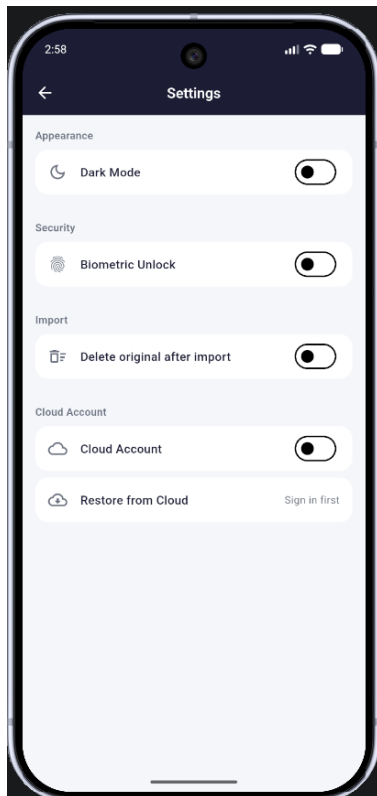
1.6.2 Vault-List

Die Vault-Übersicht zeigt alle erstellten Vaults in einer Liste an. Für jeden Vault werden wichtige Informationen wie Name, Anzahl der gespeicherten Dateien sowie der Status der Cloud-Sicherung dargestellt. Zusätzlich steht eine Löschfunktion zur Verfügung.

Durch die Auswahl eines Vaults gelangen Nutzer zur Detailansicht. Neue Vaults können über den Button „New Vault“ erstellt werden. Über das Zahnrad oben rechts können außerdem die Einstellungen geöffnet werden.



1.6.3 Settings



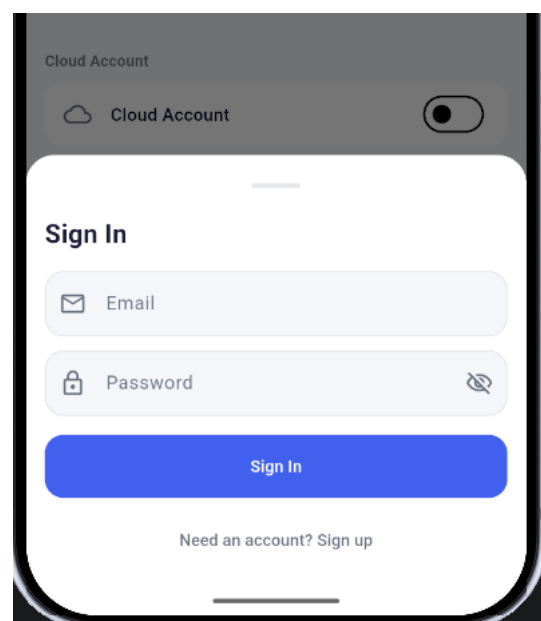
Der Einstellungsbildschirm fasst alle wichtigen Konfigurationsmöglichkeiten der Anwendung zusammen. Nutzer können hier unter anderem das Theme (Light-/Dark-Mode), biometrisches Entsperren und weitere Optionen verwalten.

Zusätzlich kann ein Cloud-Konto verbunden und bereits gesicherte Vaults aus der Cloud wiederhergestellt werden.

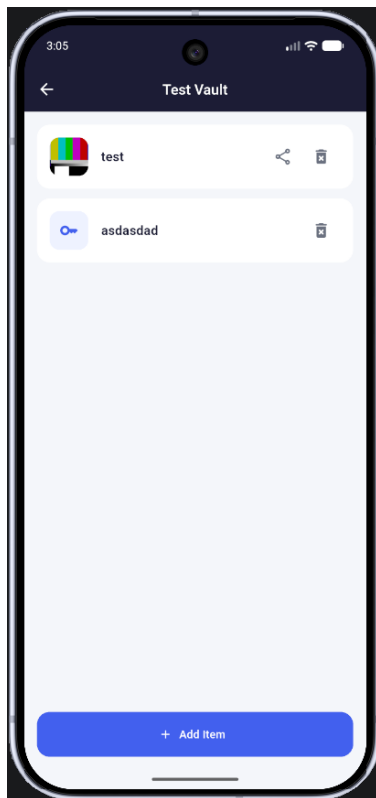
1.6.4 Cloud Log-In

Die Cloud-Anmeldung wird als Bottom-Sheet dargestellt und ermöglicht die Anmeldung oder Erstellung eines neuen Supabase-Kontos.

Der Zugriff darauf ist sowohl über die Einstellungen als auch über die Cloud-Sync-Badge möglich.



1.6.5 Vault-Details



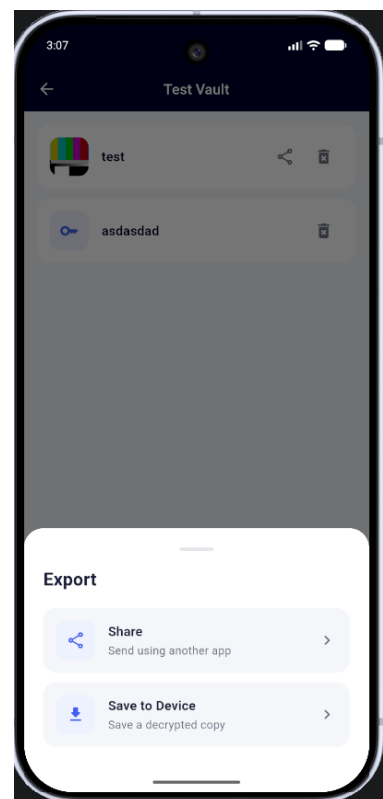
Die Detailansicht eines Vaults dient zur Verwaltung aller gespeicherten Elemente. Nutzer erhalten eine Übersicht aller enthaltenen Einträge, die verschlüsselt gespeichert und bei Bedarf entschlüsselt angezeigt werden.

Über den Button „Add Item“ können neue Elemente hinzugefügt und der entsprechende Typ ausgewählt werden. Zusätzlich stehen Funktionen zum Anzeigen, Exportieren, Teilen und Löschen einzelner Einträge zur Verfügung.

1.6.6 Export

Das Export-Menü ermöglicht die Weitergabe oder lokale Speicherung von entschlüsselten Vault-Inhalten. Nutzer können auswählen, ob ein Element über die Teilen-Funktion an andere Anwendungen gesendet oder als Kopie auf dem Gerät gespeichert werden soll.

Die Auswahl wird übersichtlich in einem Bottom-Sheet dargestellt und ermöglicht einen schnellen Zugriff auf die Exportfunktionen.



2. Tools

2.1 Programmiersprache und verwendete Version

Gecodet wurde mit dem Framework Flutter in der Programmiersprache Dart.

Verwendete Versionen zum Entwicklungszeitpunkt:

Flutter: 3.44.0 Channel Stable

Dart: 3.12.0 stable

DevTools: 2.57.0

2.2 Entwicklungsumgebungen

- IDE: Visual Studio Code; Android Studio (Device Emulator)
- Versionskontrolle: Git & GitHub
- Plattform: Android, Web (zu anfänglichen Debug zwecken)

2.3 APIs

MinVault verwendet Supabase zur Erstellung verschlüsselter Backups von Vaults.

Die Authentifizierung erfolgt über Supabase Authentication mittels E-Mail und Passwort. Zusätzlich können Benutzer ihre Vaults verschlüsselt in einem Supabase Storage hochladen um ein Backup zu ermöglichen.

Dabei verfolgt MinVault ein Zero-Knowledge-Prinzip: Dateien werden beim hinzufügen in einen Vault verschlüsselt lokal auf dem Endgerät gespeichert. Die zur Ver- und Entschlüsselung benötigten Schlüssel verlassen das Endgerät zu keinem Zeitpunkt und werden ausschließlich im sicheren Schlüsselspeicher des Betriebssystems verwaltet.

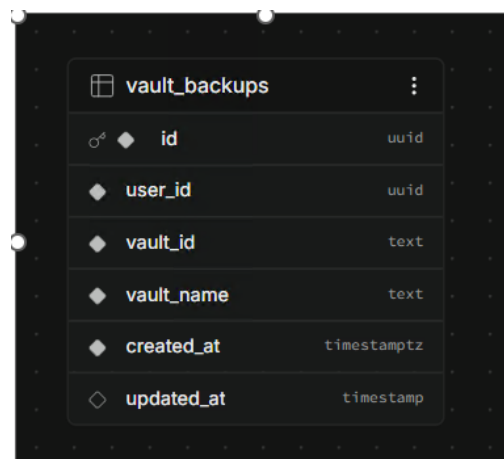
Dadurch liegen auf Supabase ausschließlich verschlüsselte Daten vor. Selbst im Falle eines unbefugten Zugriffs auf den Cloud-Speicher können die gespeicherten Vaults ohne den lokalen Schlüssel nicht entschlüsselt oder eingesehen werden.

Die Kommunikation mit Supabase erfolgt ausschließlich über verschlüsselte HTTPS-Verbindungen mithilfe des offiziellen Flutter SDKs.

2.4 Supabase

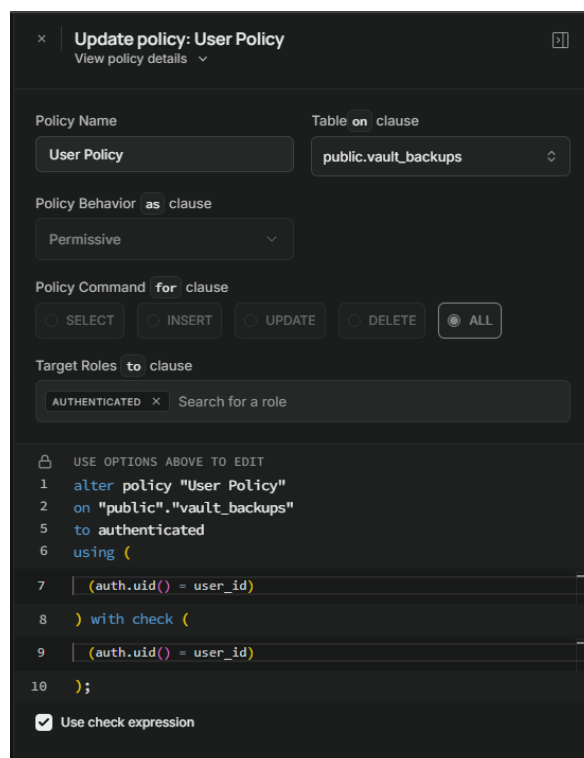
2.4.1 Datenbank

Zur Verwaltung der Cloud-Backups wird in Supabase die Tabelle „vault_backups“ verwendet. Diese speichert ausschließlich Metadaten zu den gesicherten Vaults, während die eigentlichen verschlüsselten Backup-Dateien separat im Storage-Bucket abgelegt werden. Dadurch werden sensible Inhalte nicht direkt in der relationalen Datenbank gespeichert.



vault_backups	
id	uuid
user_id	uuid
vault_id	text
vault_name	text
created_at	timestampz
updated_at	timestamp

Die Tabelle dient der Zuordnung und Nachverfolgung vorhandener Backups pro Benutzer und ermöglicht zukünftige Erweiterungen wie eine Backup-Historie, automatische Bereinigung alter Sicherungen oder eine Restore-Auswahl.



Update policy: User Policy
View policy details

Policy Name: Table on clause:

Policy Behavior as clause:

Policy Command for clause: ☐ SELECT ☐ INSERT ☐ UPDATE ☐ DELETE ☒ ALL

Target Roles to clause:

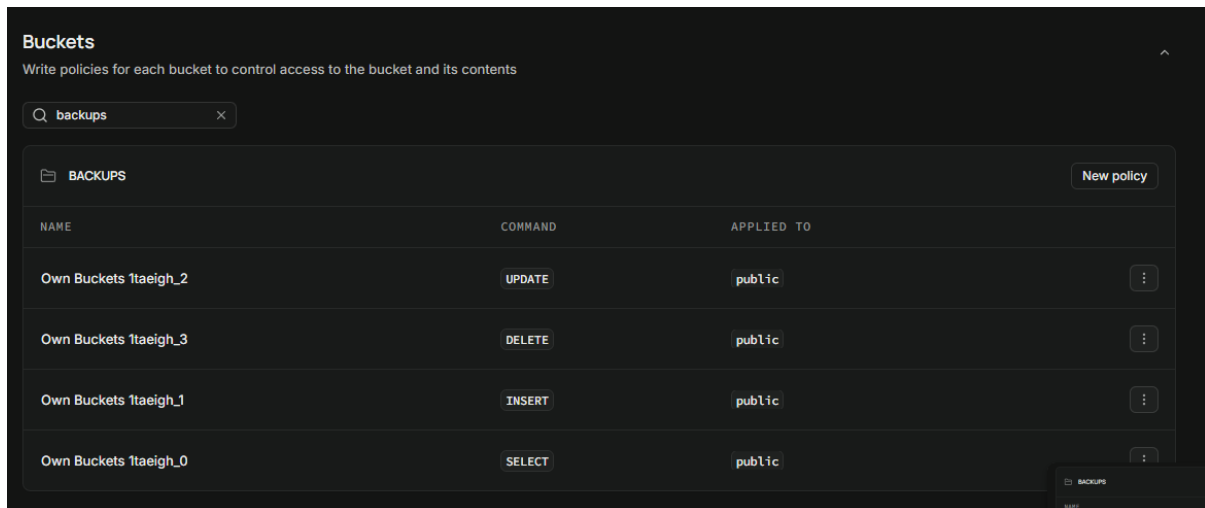
USE OPTIONS ABOVE TO EDIT

```
1 alter policy "User Policy"
2 on "public"."vault_backups"
5 to authenticated
6 using (
7   (auth.uid() = user_id)
8 ) with check (
9   (auth.uid() = user_id)
10 );
```

☒ Use check expression

2.4.2 Policies

Der Storage-Bucket „backups“ wird durch Row-Level-Security-(RLS)-Policies geschützt. Jeder Benutzer kann ausschließlich auf seine eigenen Backup-Dateien zugreifen. Die aktivierten Policies erlauben das Erstellen, Lesen, Aktualisieren und Löschen von Dateien innerhalb des eigenen Benutzerpfads.



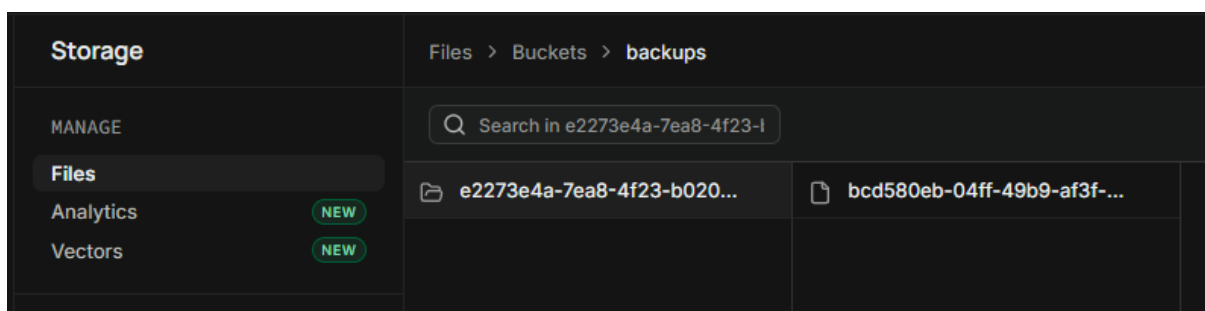
Die Zugriffskontrolle erfolgt über die Benutzer-ID im Speicherpfad (backups/{user_id}/...).

Da die Dateien zusätzlich clientseitig verschlüsselt werden und der Master-Key beziehungsweise Data Encryption Key (DEK) niemals den Client verlässt, kann selbst der Cloud-Anbieter die gespeicherten Daten nicht entschlüsseln.

2.4.3 Storage

Die eigentlichen Vault-Backups werden als ZIP-Dateien im Supabase Storage Bucket „backups“ gespeichert. Jede Datei enthält die gesicherten Vault-Daten.

Diese wurden clientseitig mit dem Data Encryption Key (DEK) des jeweiligen Vaults beim Import bereits verschlüsselt.



Die eindeutige Zuordnung erfolgt über die im Dateinamen enthaltene Vault-ID. Durch die Verwendung von Row-Level-Security-Policies wird sichergestellt, dass ausschließlich der jeweilige Besitzer Zugriff auf seine Backups erhält.

2.5 Wichtige Dependencies

Architektur & State Management

- **flutter_bloc (^9.1.1)** - Umsetzung des BLoC/Cubit-Patterns für das State Management
- **equatable (^2.0.8)** - Vereinfachter Vergleich von States und Modellen
- **get_it (^9.2.1)** - Dependency Injection und Service Locator
- **go_router (^17.2.3)** - Deklaratives Routing innerhalb der Anwendung

Sicherheit & Verschlüsselung

- **cryptography (^2.9.0)** - Kryptographische Verfahren (u. a. AES-GCM)
- **flutter_secure_storage (^10.3.1)** - Sichere Speicherung sensibler Schlüssel im Android Keystore bzw. iOS Keychain
- **local_auth (^3.0.2)** - Biometrische Authentifizierung (Fingerprint und Face Unlock)

Authentifizierung

- **supabase_flutter (^2.16.0)** - Anbindung an Supabase Authentication und Benutzerverwaltung

Lokale Datenhaltung

- **path_provider (^2.1.5)** - Zugriff auf App-spezifische Speicherorte
- **path (^1.9.1)** - Plattformunabhängige Dateipfade
- **shared_preferences (^2.5.5)** - Speicherung nicht sicherheitskritischer Benutzereinstellungen

Sonstige Bibliotheken

- **uuid (^4.6.0)** - Erzeugung eindeutiger Identifikatoren
- **cupertino_icons (^1.0.8)** - iOS-Icons für Flutter
- **archive: (^4.0.9)** - Erzeugen von Zip dateien für Cloud-Backup und geplanten Vault-Export

Testing

- **flutter_test** - Unit- und Widget-Tests
- **bloc_test (^10.0.0)** - Testen von Cubits und BLoCs
- **mocktail (^1.0.5)** - Erstellung von Mock-Objekten für Unit-Tests

Geplante Features

- **share_plus ^13.2.1** - Sicheres Exportieren und Teilen verschlüsselter Tresor-Dateien (Vault Export)

2.6 Quellen und Lizenzen

Grafische Assets

- App-Logo wurde eigenständig erstellt.
- Lizenz: Eigene Urheberschaft.

Icons

- Material Icons – Apache License 2.0 (Bestandteil von Flutter)
- Cupertino Icons (^1.0.8) – MIT License

Open-Source Bibliotheken

Alle verwendeten Bibliotheken stammen von **pub.dev** und werden entsprechend ihrer jeweiligen Open-Source-Lizenzen (z. B. MIT, BSD oder Apache 2.0) verwendet.

3. Projektarchitektur

3.1 Beschreibung der gesamten Architektur

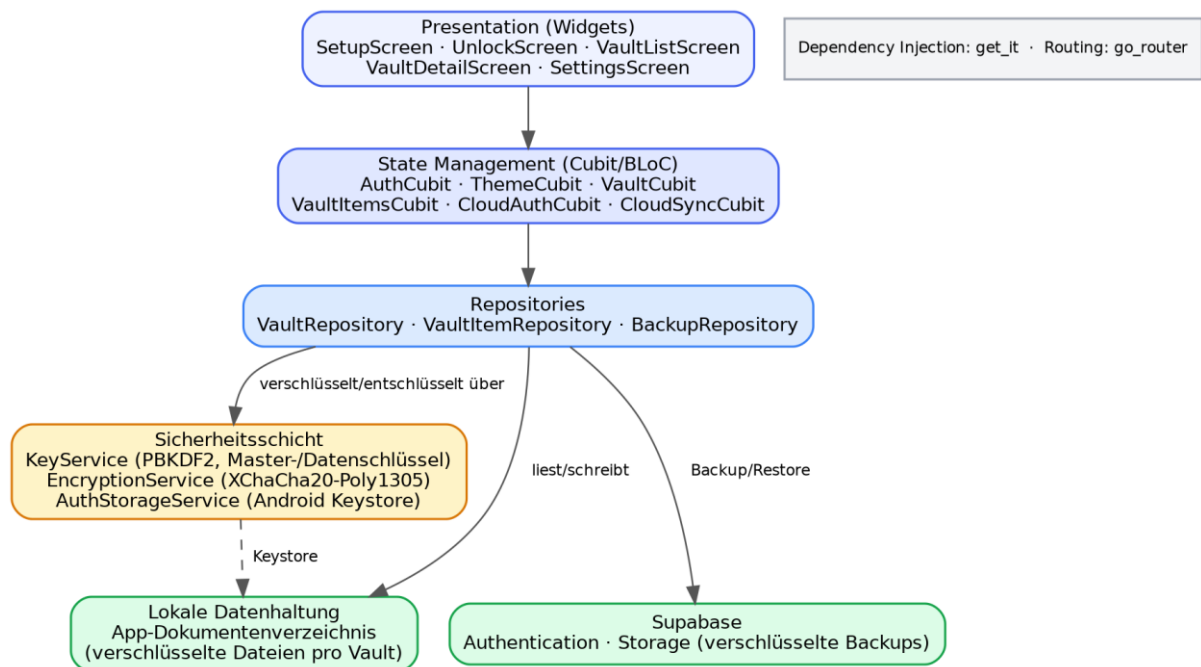


ABBILDUNG 2: PROJEKT ARCHITEKTUR VON MINVAULT

MinVault folgt einer feature-orientierten “Vertical-Slice Architektur” mit klarer Trennung von Darstellung, Zustandsverwaltung, Datenzugriff und Sicherheitslogik.

Die Präsentationsschicht besteht aus den Screens, die ausschließlich über Cubits mit dem Rest der Anwendung kommunizieren.

Die Cubits greifen ihrerseits auf Repositories zu, die für die Datenhaltung zuständig sind. Dependency Injection erfolgt zentral über “get_it”.

Eine eigene Sicherheitsschicht die für kryptographische Operationen sowie den Zugriff auf den Android Keystore zuständig ist, wird von den Repositories bei jedem Lese- und Schreibzugriff verwendet.

Die Navigation ist deklarativ über “go_router” umgesetzt worden.

Sie ist Zustandsabhängig basierend darauf, ob der Benutzer derzeit angemeldet ist, oder eventuell ein initiales Setup erforderlich ist.

3.2 Use-Case Diagramm

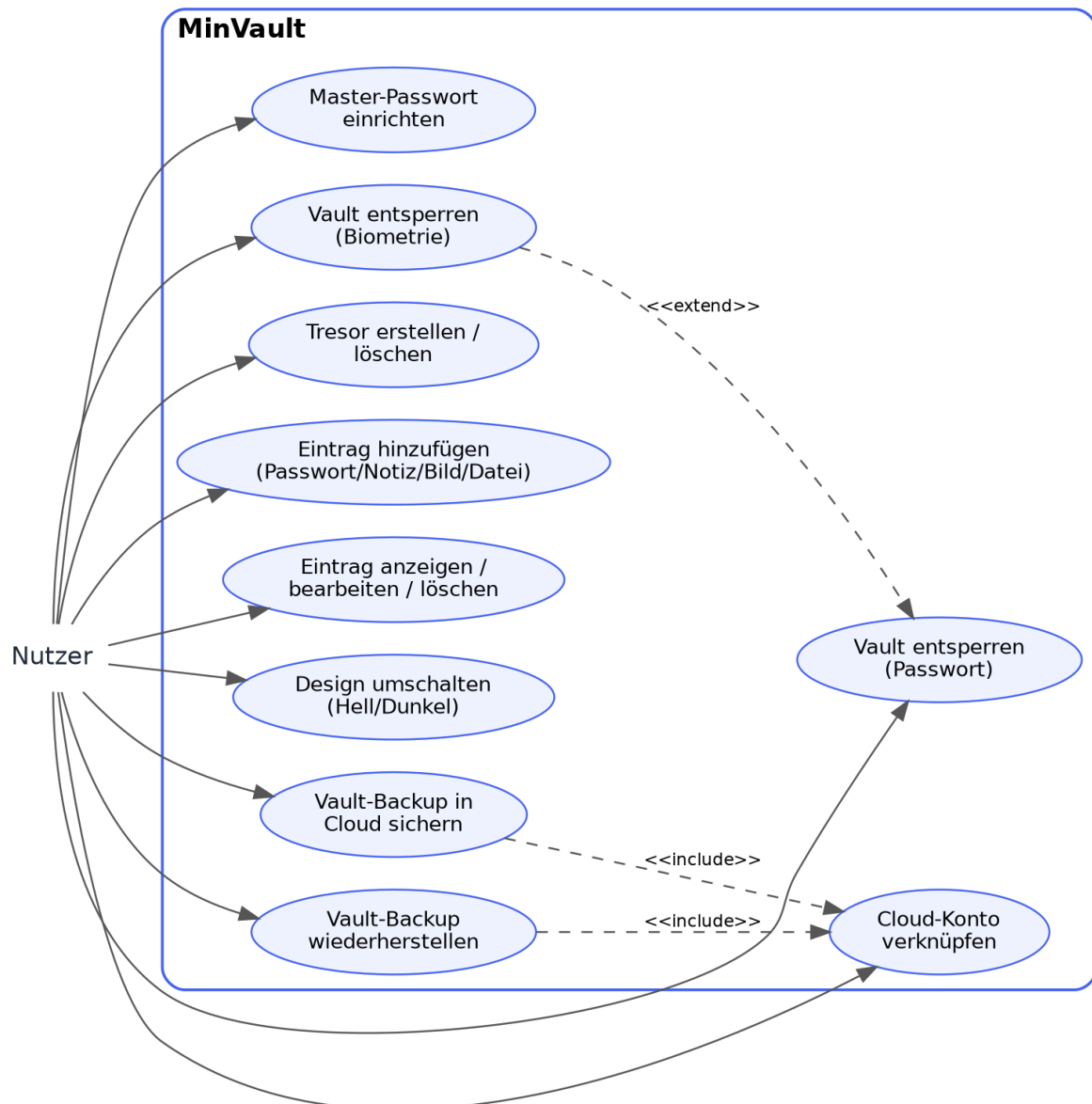


ABBILDUNG 3: USE-CASE DIAGRAMM VON MINVAULT

3.3 Klassendiagramm

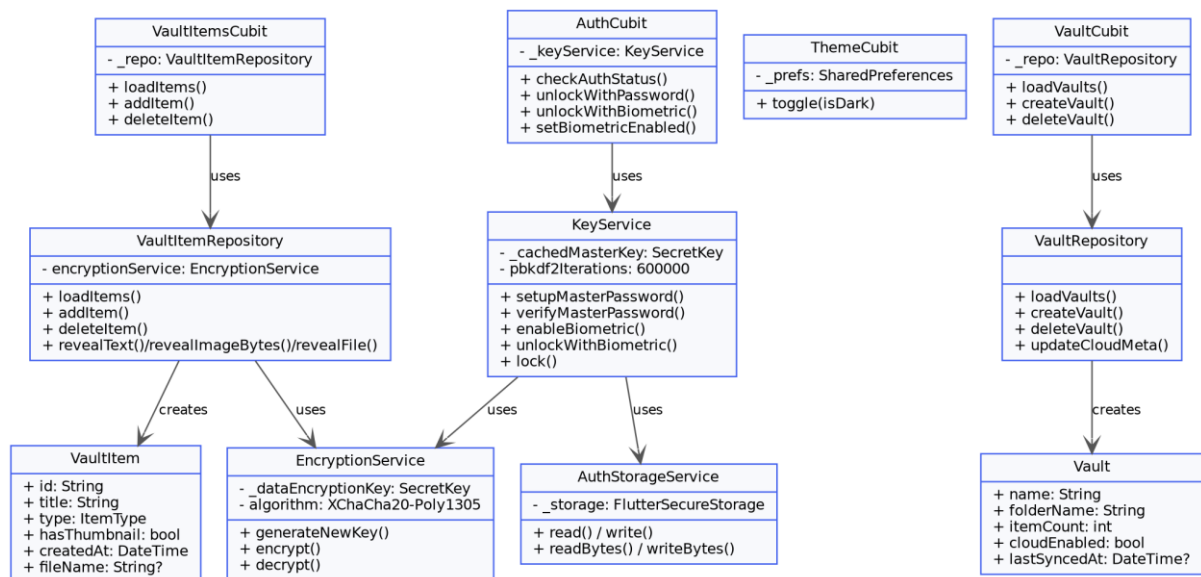


ABBILDUNG 4: KLASSENDIAGRAMM DER VAULTS

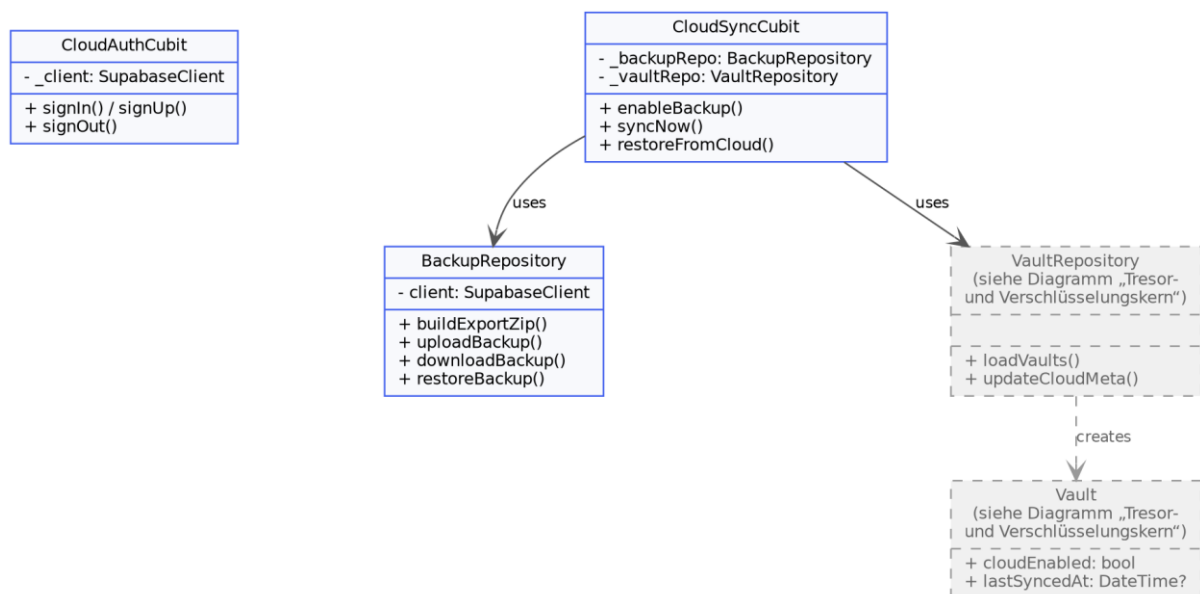


ABBILDUNG 5: KLASSENDIAGRAMM DER CLOUD-BACKUPS

4. Testen

4.1 Unit Tests

Die Anwendung enthält 8 automatisierte Unit-Tests in zwei Klassen, umgesetzt mit “flutter_test”, “bloc_test” und “mocktail”

VaultCubitTest (5 Testfälle)

- Initialer Zustand ist VaultInitial
- loadVaults() erzeugt die Zustandsfolge [VaultLoading, VaultLoaded] bei Erfolg
- loadVaults() erzeugt die Zustandsfolge [VaultLoading, VaultError] bei Fehler
- createVault() delegiert an das Repository und lädt die Liste anschließend neu
- deleteVault() delegiert an das Repository und lädt die Liste anschließend neu

EncryptionServiceTest (3 Testfälle)

- Verschlüsseln und anschließendes Entschlüsseln liefert den ursprungstext zurück
- Entschlüsseln mit einem falschen Schlüssel schlägt kontrolliert fehl (Exception)
- Zwei Verschlüsselungen desselben Inputs erzeugen unterschiedliche Chiffre, da ein neuer zufälliger Nonce verwendet wird

```
• E:\Programming\Repositorys\Uni\502-EmA\min_vault > main flutter test
Resolving dependencies...
Downloading packages...
  _fe_analyzer_shared 99.0.0 (105.0.0 available)
  analyzer 12.1.0 (14.1.0 available)
  cli_util 0.4.2 (0.5.2 available)
  dbus 0.7.13 (0.7.14 available)
  file_picker 10.3.10 (11.0.2 available)
  flutter_secure_storage_darwin 0.3.2 (0.4.0 available)
  flutter_secure_storage_platform_interface 2.0.1 (2.0.2 available)
  flutter_secure_storage_windows 4.1.0 (4.2.2 available)
  hooks 2.0.2 (2.1.0 available)
  matcher 0.12.19 (0.12.20 available)
  meta 1.18.0 (1.19.0 available)
  package_config 2.2.0 (3.0.0 available)
  posix 6.5.0 (6.5.2 available)
  record_use 0.6.0 (1.0.0 available)
  share_plus 12.0.2 (13.3.0 available)
  share_plus_platform_interface 6.1.0 (7.2.0 available)
  test 1.31.0 (1.31.2 available)
  test_api 0.7.11 (0.7.13 available)
  test_core 0.6.17 (0.6.19 available)
  vector_math 2.2.0 (2.4.1 available)
  win32 5.15.0 (6.3.0 available)
Got dependencies!
21 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
00:01 +8: All tests passed!
```

4.2 Manuelle Testfälle

Testfall	Vorgehen	Erwartetes Ergebnis
Ersteinrichtung	App im Erstzustand starten, Master-Passwort vergeben.	Setup-Screen erscheint einmalig, anschließend direkter Wechsel zur Vault-Übersicht.
Falsches Passwort	Nach dem Sperren ein falsches Passwort eingeben.	Fehlermeldung wird angezeigt, App bleibt gesperrt, kein Datenschlüssel wird geladen.
Biometrische Entsperrung	Biometrie in den Einstellungen aktivieren, App sperren und per Fingerabdruck entsperren.	App entsperrt sich nach erfolgreicher biometrischer Prüfung, ohne erneute Passwordeingabe.
Design umschalten	In den Einstellungen zwischen Hell- und Dunkel-Design wechseln.	Design der gesamten App ändert sich sofort und bleibt nach Neustart erhalten.
Tresor anlegen/löschen	Neuen Tresor erstellen, anschließend wieder löschen.	Tresor erscheint sofort in der Übersicht bzw. verschwindet nach dem Löschen inkl. aller Inhalte.
Passwort-Eintrag	Passwort-Eintrag in einem Tresor anlegen, öffnen.	Eintrag wird verschlüsselt gespeichert, ist nach dem Entsperren korrekt lesbar.
Bild-Eintrag	Bild aus der Galerie zu einem Tresor hinzufügen.	Verschlüsseltes Thumbnail erscheint in der Liste, Originalbild ist nach dem Öffnen unverändert sichtbar.
Cloud-Anmeldung	Mit dem Test-Account bei Supabase anmelden und abmelden.	Login-Status wird korrekt als Schalter in den Einstellungen angezeigt, kein separater Anmelde-Button sichtbar.
Cloud-Backup	Tresor nach erfolgter Anmeldung in die Cloud sichern.	Zip mit bereits verschlüsselten Dateien wird hochgeladen; Backup-Status wird pro Tresor angezeigt.

4.3 Häufige Probleme und Lösungen

Problem	Beschreibung	Lösung / Status
Statischer PBKDF2-Salt	In einer frühen Version wurde für die Schlüsselableitung ein fest im Quellcode hinterlegter Salt verwendet, wodurch der Salt bei jeder Installation identisch gewesen wäre.	Umstellung auf einen pro Installation zufällig generierten 16-Byte-Salt (<code>Random.secure()</code>), der verschlüsselt im Secure Storage abgelegt und bei der Passwortprüfung wieder geladen wird.
Layout-Verschiebung	Der Biometrie-Button auf dem UnlockScreen erschien mit spürbarer Verzögerung und verschob dadurch das restliche Layout.	Der Biometrie-Status wird jetzt synchron beim Aufbau des Screens aus dem bereits vorliegenden Auth-Zustand gelesen statt erst asynchron nachgeladen, wodurch kein Zwischenzustand mehr gerendert wird.
UI überlappt mit Chome	Auf Geräten mit reiner Gestennavigation (getestet auf einem Nothing Phone) überlappte die Navigationsleiste den „Neuer Tresor“-Button am unteren Bildschirmrand.	Einbindung in ein SafeArea-Widget, sodass Bedienelemente auf allen getesteten Geräten erreichbar bleiben.
file_picker liefert gecachten Pfad	Für importierte Dateien liefert das file_picker-Plugin plattformbedingt nur eine zwischengespeicherte Kopie, keinen Zugriff auf den ursprünglichen Speicherort.	Bewusst als bekannte Einschränkung akzeptiert; das automatische Löschen der Originaldatei nach dem Import wurde deshalb im Rahmen des Zeitbudgets zurückgestellt.
Entschlüsselte Temp-Dateien beim Anzeigen	Beim Anzeigen/Teilen von Bild- und Dateieinträgen wird die entschlüsselte Datei temporär im Cache-Verzeichnis abgelegt.	Aktuell keine automatische Löschung nach Gebrauch; als bekannte Einschränkung dokumentiert und für eine spätere Bereinigung beim Sperren der App vorgesehen.
Geräte übergreifende Backup-Recovery	Da der Encryption-Key pro Installation zufällig erzeugt wird, kann ein auf einem anderen Gerät wiederhergestelltes Backup nicht entschlüsselt werden.	Als architekturelle Einschränkung erkannt und dokumentiert; eine passwortabgeleitete oder synchronisierte Schlüssel-Lösung ist als Erweiterung vorgesehen.

5. Aktueller Stand

Die MinVault-App läuft stabil und ist mit einem modernen UI versehen. Es wurden alle geplanten Hauptfunktionen (siehe User-Stories) umgesetzt:

Nutzer legen beim ersten Starten der App ein Master-Passwort fest, welches jedes mal beim Neustart der App eingegeben werden muss, um Zugang zu den Vaults zu bekommen. Alternativ kann anschließend, falls der Nutzer lieber mit einem Fingerabdruck die App entsperren möchte, die Einstellung für biometrisches Entsperren aktiviert werden. In der Vault-Liste können neue Vaults erstellt/gelöscht werden. Diese Vaults sind obfuskierte Verzeichnisse im lokalen Speicher. Sobald in den Einstellungen die Option "Cloud Account" aktiviert ist und man sich registriert hat/eingeloggt hat, kann man für jeden Vault die Cloud-Backup Funktion aktivieren. Hierbei wird ein Backup (zip-Datei) an den Supabase Storage geschickt. In der Liste gibt jeder Vault an, wie viele Einträge (Items in Form von Passwörtern, Notizen, Bildern und anderen Dokumenten) sich jeweils darin befinden.

In der Vault-Items-Liste wird der Inhalt eines Vaults angezeigt. Hier kann ein Nutzer neue Einträge hinzufügen, welche verschlüsselt im lokalen Vault-Verzeichnis gespeichert werden. Passwörter und Notizen werden direkt in der App erstellt. Bilder und andere Dokumente werden mit Hilfe des `file_picker` packages, direkt vom Gerätespeicher geholt. Diese können über den Share-Button, entweder über eine andere App versendet, oder wieder unverschlüsselt auf dem Gerät gespeichert werden. Standardmäßig verbleibt die Originaldatei im Gerätespeicher, nachdem sie einem Vault hinzugefügt wurde. Wenn ein Nutzer das nicht möchte, kann er in den Einstellungen die Option "Delete original after import" aktivieren.

Zudem kann in den Einstellung ein Darkmode aktiviert werden.

6. Ausblick

Obwohl sich die App in einem stabilen Zustand befindet und alle geplanten Hauptfunktionen umgesetzt wurden, gibt es schon Pläne zur Erweiterung.

Ein sinnvolles Feature wäre die Möglichkeit Vaults zu exportieren oder zu teilen (siehe User Story US08). Hierdurch könnte der Nutzer einen Vault in Form einer verschlüsselten Datei exportieren und nach Belieben nutzen oder diese über andere Apps teilen.

Desweiteren wäre es nützlich bestehende Passworteinträge und Notizen zu bearbeiten, damit kein neuer Eintrag angelegt werden muss. Den Inhalt einer Notiz oder eines Passworts kann zwar kopiert werden und dadurch kann ein Nutzer leicht einen neuen Eintrag anfertigen, allerdings ist dies ein Extraschritt der getätigt werden muss.

Zusätzlich können noch Features zur Barrierefreiheit implementiert werden, wie ein Screen-Reader, der Textinhalte in der App vorliest, Themes mit hohem Farbkontrast und skalierbare Schriftarten.

Zuletzt könnte man eine zusätzliche Schutzebene bereitstellen, indem man eine Root- oder Jailbreak-Erkennung implementiert.

6.1 Weiterentwicklung

Für die Weiterentwicklung ist es wichtig folgende Punkte zu beachten:

Beim Teilen oder Öffnen von Einträgen, werden die temporär entschlüsselten Dateien im Cache zwischengespeichert, aber bisher nicht bereinigt. Dadurch sind diese Dateien nicht mehr geschützt.

Beim Importieren von Dateien kann die Originaldatei derzeit nicht automatisch gelöscht werden, da das `file_picker` Plugin derzeit einen gecachten Pfad zurückliefert.

Zur Zeit wird der Data-Encryption-Key (DEK), also der Schlüssel der genutzt wird um die Vault-Einträge zu verschlüsseln, pro Installation zufällig erzeugt, statt aus dem Master-Passwort abgeleitet, wodurch eine Wiederherstellung nur auf dem Ursprungsgerät möglich ist.

Zudem wird der beim Entsperren mit einem Fingerabdruck verwendete Key vor dem Auslesen geprüft, ist aber nicht an ein `CryptoObject` (genutzt von Android Keystore Biometrie API) gebunden, was die Sicherheit etwas vermindert.

7. Fazit

Bei der Umsetzung der MinVault-App haben wir uns intensiv mit Themen wie Kryptographie, Flutter-Architektur, State Management, Umgang mit Nutzerdaten und Cloud-Anbindung auseinandergesetzt. Besonderer Fokus lag auf der Entscheidung zwei Schlüsselebenen zu verwenden (Envelope Encryption, DEK-KEK-Architektur), wobei ein Schlüssel vom Master-Passwort abgeleitet wird und ein weiterer Schlüssel die Daten verschlüsselt. Des weiteren war die Auswahl des Verschlüsselungs-Algorithmus selbst sehr wichtig, da die Performance abhängig von der gegebenen Hardware ist, und gerade bei mobilen Geräten so etwas wie Akkuverbrauch beachtet werden muss. Somit fiel die Wahl auf den Chacha20-poly1305 Algorithmus, der sich auf ARM-Hardware bewährt.

Da das Zeitbudget beachtet werden musste wurde nicht jede Maßnahme, um die Sicherheit zu maximieren (z.B. Hardware-gebundene Biometrie-Schlüssel oder Code-Obfuskierung), umgesetzt. Deshalb wurden diese Punkte in Kapitel 6 als Ausblick festgehalten.

Insgesamt hat uns das Projekt gezeigt, dass gute Sicherheit weniger eine einzelne Funktion als vielmehr eine durchgängige Designentscheidung ist, die von der Datenhaltung bis zur UI reicht.

Letztendlich hatten wir Spaß und haben praktische Erfahrung für die Entwicklung mobiler Anwendungen gesammelt und können damit zukünftige App-Ideen leichter und schneller umsetzen.

8. Quellen

- <https://moodle.hs-worms.de/course/view.php?id=3580>
- <https://docs.flutter.dev/ui>
- <https://dart.dev/docs>
- <https://docs.phase.dev/security/cryptography>
- <https://soatok.blog/2020/07/12/comparison-of-symmetric-encryption-methods/#aes-gcm-vs-chacha20poly1305>
- <https://shattered.io/chacha20-poly1305-vs-aes-256-gcm/>
- <https://www.newsoftwares.net/blog/authenticated-encryption-aead-stopping-tampering/>
- https://pub.dev/packages/go_router
- https://pub.dev/packages/get_it
- https://pub.dev/packages/file_picker
- <https://pub.dev/packages/cryptography>
- <https://supabase.com/docs/guides/getting-started/quickstarts/flutter>